

v2

Threat Hunting Professional

Introduction to Malware Hunting

Section 03 | Module 02

© Caendra Inc. 2020
All Rights Reserved

Table of Contents

MODULE 02 | INTRODUCTION TO MALWARE HUNTING

2.1 Introduction

2.2 Malware Classifications

2.3 Malware Delivery

2.4 Malware Evasion Techniques

2.5 Malware Persistence

```
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

Introduction



2.1 Introduction

What is malware?

Malware is the short form of **malicious software**.

It is software written to infiltrate or damage a computer system without the owner's consent. It can be considered one of the following: intrusive, hostile, and/or annoying.

2.1 Introduction

We will not focus on annoying malware, such as adware and/or PUPs (Potentially Unwanted Programs).

We will be looking at intrusive and hostile forms of malware.



Malware Classifications



2.2.1 Viruses

A computer **virus** is a program that copies itself and spreads without the permission or knowledge of the owner. Viruses do not spread via exploiting vulnerabilities (the ones that do that are called **worms**).

The only way viruses are supposed to spread is with the host, at least in their rigorous classification. Let's say that a virus has infected a file; now if the owner moves the file to a different system, the virus has a chance to spread and survive.

2.2.1 Viruses

Viruses can be classified into the following sub-types:

- Resident
- Non-resident
- Boot Sector
- Multi-Partite

```
21 def initialize(experiment, observations = [], control = null)
22   # @param: experiment - the experiment this result is for
23   # @param: observations - an array of Observations, in ascending order
24   # @param: control - the control observation
25
26   @experiment = experiment
27   @observations = observations
28   @control = control
29   @candidates = observations - [control]
30   evaluate_candidates
31
32   freeze
33 end
34
35 # @param: the experiment's context
36 def context
37   @experiment.context
38 end
39
40 # @param: the name of the experiment
41 def experiment_name
42   @experiment.name
43 end
44
45 # @param: the result a match between all candidates
46 def match
47   @result
48 end
49
50 def match
51   @result
52 end
53
54 def result
55   @result
56 end
57
58 def result
59   @result
60 end
61
62 def result
63   @result
64 end
65
66 def result
67   @result
68 end
69
70 def result
71   @result
72 end
73
74 def result
75   @result
76 end
77
78 def result
79   @result
80 end
81
82 def result
83   @result
84 end
85
86 def result
87   @result
88 end
89
90 def result
91   @result
92 end
93
94 def result
95   @result
96 end
97
98 def result
99   @result
100 end
101
102 def result
103   @result
104 end
105
106 def result
107   @result
108 end
109
110 def result
111   @result
112 end
113
114 def result
115   @result
116 end
117
118 def result
119   @result
120 end
121
122 def result
123   @result
124 end
125
126 def result
127   @result
128 end
129
130 def result
131   @result
132 end
133
134 def result
135   @result
136 end
137
138 def result
139   @result
140 end
141
142 def result
143   @result
144 end
145
146 def result
147   @result
148 end
149
150 def result
151   @result
152 end
153
154 def result
155   @result
156 end
157
158 def result
159   @result
160 end
161
162 def result
163   @result
164 end
165
166 def result
167   @result
168 end
169
170 def result
171   @result
172 end
173
174 def result
175   @result
176 end
177
178 def result
179   @result
180 end
181
182 def result
183   @result
184 end
185
186 def result
187   @result
188 end
189
190 def result
191   @result
192 end
193
194 def result
195   @result
196 end
197
198 def result
199   @result
200 end
201
202 def result
203   @result
204 end
205
206 def result
207   @result
208 end
209
210 def result
211   @result
212 end
213
214 def result
215   @result
216 end
217
218 def result
219   @result
220 end
221
222 def result
223   @result
224 end
225
226 def result
227   @result
228 end
229
230 def result
231   @result
232 end
233
234 def result
235   @result
236 end
237
238 def result
239   @result
240 end
241
242 def result
243   @result
244 end
245
246 def result
247   @result
248 end
249
250 def result
251   @result
252 end
253
254 def result
255   @result
256 end
257
258 def result
259   @result
260 end
261
262 def result
263   @result
264 end
265
266 def result
267   @result
268 end
269
270 def result
271   @result
272 end
273
274 def result
275   @result
276 end
277
278 def result
279   @result
280 end
281
282 def result
283   @result
284 end
285
286 def result
287   @result
288 end
289
290 def result
291   @result
292 end
293
294 def result
295   @result
296 end
297
298 def result
299   @result
300 end
301
302 def result
303   @result
304 end
305
306 def result
307   @result
308 end
309
310 def result
311   @result
312 end
313
314 def result
315   @result
316 end
317
318 def result
319   @result
320 end
321
322 def result
323   @result
324 end
325
326 def result
327   @result
328 end
329
330 def result
331   @result
332 end
333
334 def result
335   @result
336 end
337
338 def result
339   @result
340 end
341
342 def result
343   @result
344 end
345
346 def result
347   @result
348 end
349
350 def result
351   @result
352 end
353
354 def result
355   @result
356 end
357
358 def result
359   @result
360 end
361
362 def result
363   @result
364 end
365
366 def result
367   @result
368 end
369
370 def result
371   @result
372 end
373
374 def result
375   @result
376 end
377
378 def result
379   @result
380 end
381
382 def result
383   @result
384 end
385
386 def result
387   @result
388 end
389
390 def result
391   @result
392 end
393
394 def result
395   @result
396 end
397
398 def result
399   @result
400 end
401
402 def result
403   @result
404 end
405
406 def result
407   @result
408 end
409
410 def result
411   @result
412 end
413
414 def result
415   @result
416 end
417
418 def result
419   @result
420 end
421
422 def result
423   @result
424 end
425
426 def result
427   @result
428 end
429
430 def result
431   @result
432 end
433
434 def result
435   @result
436 end
437
438 def result
439   @result
440 end
441
442 def result
443   @result
444 end
445
446 def result
447   @result
448 end
449
450 def result
451   @result
452 end
453
454 def result
455   @result
456 end
457
458 def result
459   @result
460 end
461
462 def result
463   @result
464 end
465
466 def result
467   @result
468 end
469
470 def result
471   @result
472 end
473
474 def result
475   @result
476 end
477
478 def result
479   @result
480 end
481
482 def result
483   @result
484 end
485
486 def result
487   @result
488 end
489
490 def result
491   @result
492 end
493
494 def result
495   @result
496 end
497
498 def result
499   @result
500 end
501
502 def result
503   @result
504 end
505
506 def result
507   @result
508 end
509
510 def result
511   @result
512 end
513
514 def result
515   @result
516 end
517
518 def result
519   @result
520 end
521
522 def result
523   @result
524 end
525
526 def result
527   @result
528 end
529
530 def result
531   @result
532 end
533
534 def result
535   @result
536 end
537
538 def result
539   @result
540 end
541
542 def result
543   @result
544 end
545
546 def result
547   @result
548 end
549
550 def result
551   @result
552 end
553
554 def result
555   @result
556 end
557
558 def result
559   @result
560 end
561
562 def result
563   @result
564 end
565
566 def result
567   @result
568 end
569
570 def result
571   @result
572 end
573
574 def result
575   @result
576 end
577
578 def result
579   @result
580 end
581
582 def result
583   @result
584 end
585
586 def result
587   @result
588 end
589
590 def result
591   @result
592 end
593
594 def result
595   @result
596 end
597
598 def result
599   @result
600 end
601
602 def result
603   @result
604 end
605
606 def result
607   @result
608 end
609
610 def result
611   @result
612 end
613
614 def result
615   @result
616 end
617
618 def result
619   @result
620 end
621
622 def result
623   @result
624 end
625
626 def result
627   @result
628 end
629
630 def result
631   @result
632 end
633
634 def result
635   @result
636 end
637
638 def result
639   @result
640 end
641
642 def result
643   @result
644 end
645
646 def result
647   @result
648 end
649
650 def result
651   @result
652 end
653
654 def result
655   @result
656 end
657
658 def result
659   @result
660 end
661
662 def result
663   @result
664 end
665
666 def result
667   @result
668 end
669
670 def result
671   @result
672 end
673
674 def result
675   @result
676 end
677
678 def result
679   @result
680 end
681
682 def result
683   @result
684 end
685
686 def result
687   @result
688 end
689
690 def result
691   @result
692 end
693
694 def result
695   @result
696 end
697
698 def result
699   @result
700 end
701
702 def result
703   @result
704 end
705
706 def result
707   @result
708 end
709
710 def result
711   @result
712 end
713
714 def result
715   @result
716 end
717
718 def result
719   @result
720 end
721
722 def result
723   @result
724 end
725
726 def result
727   @result
728 end
729
730 def result
731   @result
732 end
733
734 def result
735   @result
736 end
737
738 def result
739   @result
740 end
741
742 def result
743   @result
744 end
745
746 def result
747   @result
748 end
749
750 def result
751   @result
752 end
753
754 def result
755   @result
756 end
757
758 def result
759   @result
760 end
761
762 def result
763   @result
764 end
765
766 def result
767   @result
768 end
769
770 def result
771   @result
772 end
773
774 def result
775   @result
776 end
777
778 def result
779   @result
780 end
781
782 def result
783   @result
784 end
785
786 def result
787   @result
788 end
789
790 def result
791   @result
792 end
793
794 def result
795   @result
796 end
797
798 def result
799   @result
800 end
801
802 def result
803   @result
804 end
805
806 def result
807   @result
808 end
809
810 def result
811   @result
812 end
813
814 def result
815   @result
816 end
817
818 def result
819   @result
820 end
821
822 def result
823   @result
824 end
825
826 def result
827   @result
828 end
829
830 def result
831   @result
832 end
833
834 def result
835   @result
836 end
837
838 def result
839   @result
840 end
841
842 def result
843   @result
844 end
845
846 def result
847   @result
848 end
849
850 def result
851   @result
852 end
853
854 def result
855   @result
856 end
857
858 def result
859   @result
860 end
861
862 def result
863   @result
864 end
865
866 def result
867   @result
868 end
869
870 def result
871   @result
872 end
873
874 def result
875   @result
876 end
877
878 def result
879   @result
880 end
881
882 def result
883   @result
884 end
885
886 def result
887   @result
888 end
889
890 def result
891   @result
892 end
893
894 def result
895   @result
896 end
897
898 def result
899   @result
900 end
901
902 def result
903   @result
904 end
905
906 def result
907   @result
908 end
909
910 def result
911   @result
912 end
913
914 def result
915   @result
916 end
917
918 def result
919   @result
920 end
921
922 def result
923   @result
924 end
925
926 def result
927   @result
928 end
929
930 def result
931   @result
932 end
933
934 def result
935   @result
936 end
937
938 def result
939   @result
940 end
941
942 def result
943   @result
944 end
945
946 def result
947   @result
948 end
949
950 def result
951   @result
952 end
953
954 def result
955   @result
956 end
957
958 def result
959   @result
960 end
961
962 def result
963   @result
964 end
965
966 def result
967   @result
968 end
969
970 def result
971   @result
972 end
973
974 def result
975   @result
976 end
977
978 def result
979   @result
980 end
981
982 def result
983   @result
984 end
985
986 def result
987   @result
988 end
989
990 def result
991   @result
992 end
993
994 def result
995   @result
996 end
997
998 def result
999   @result
1000 end
```

2.2.1 Viruses

Resident

When the virus is executed and becomes memory resident. It waits for some trigger, such as loading another program. It then infects other programs and so on.

Non-Resident

When the virus is executed, it will search for files it can infect. After infecting them, it will quit. When the infected program is run again, it will again find new targets to infect and so on.

2.2.1 Viruses

Boot Sector

Spreads via boot sectors. For example, if a user leaves an infected CD-ROM in the drive while turning off a system. The next time the system boots up, the boot sector virus will activate and will spread to the hard disk, which will then spread to USB flash drives. When the flash drives are moved, the cycle repeats.

Multi-Partite

These viruses have several types of infection mechanisms. They can have both **Boot-Sector** and **Resident** type viruses or even more.

2.2.2 Worm

Worms are basically software which uses network/system vulnerabilities to spread themselves from system to system. They are typically part of other software, such as rootkits, and are normally the entry point into the system. They basically compromise the system (locally or remotely) and provide access to other malware.

2.2.3 Rootkits

A **rootkit** is malware which is designed to hide the fact that a compromise has occurred, or to compromise the system at a deeper level. A rootkit is basically used as a supplement to other malware.

Rootkits can be used to:

- Hide processes
- Add files to the file system
- Implement backdoors
- Create loopholes

```
24
25
26 # @param self the Experiment class itself
27 # @param observations an array of Observations, or None
28 # @param control the control Observation
29
30 def initialize(self, observations=None, control=None):
31     @experiment = self
32     @observations = observations
33     @control = control
34     @candidates = []
35     evaluate_candidates()
36
37 freeze
38
39 # @param self the Experiment's context
40 def context(self):
41     return self._context
42
43 # @param self the name of the experiment
44 def experiment_name(self):
45     return self._experiment_name
46
47 # @param self the results a match between self and
48 def match(self):
49     return self._match
50
51 # @param self the results a match between self and
```



2.2.3 Rootkits

When a rootkit is installed, the entire operating system is compromised. Rootkits exist for all major operating systems. They are installed as drivers (or kernel modules).

They are known to exist at the following levels (even lower levels are possible):

- Application Level
- Library Level
- Kernel Level
- Hypervisor Level
- Firmware Level

2.2.3 Rootkits

Application Level

They actually replace programs with copies of other programs.

Library Level

Let's say that 10 applications are sharing a library. Taking control of the library means taking control of all 10 applications.

```
20 def initialize(experiment, observations = [], control = null)
21   @experiment = experiment
22   @observations = observations
23   @control = control
24   @candidates = []
25   evaluate_candidates
26 end
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = []
33   evaluate_candidates
34 end
35
36 def freeze
37   @candidates = []
38 end
39
40 def freeze
41   @candidates = []
42 end
43
44 def match
45   @candidates = []
46 end
47
48 def match
49   @candidates = []
50 end
51
52 def match
53   @candidates = []
54 end
55
56 def match
57   @candidates = []
58 end
59
60 def match
61   @candidates = []
62 end
63
64 def match
65   @candidates = []
66 end
67
68 def match
69   @candidates = []
70 end
71
72 def match
73   @candidates = []
74 end
75
76 def match
77   @candidates = []
78 end
79
80 def match
81   @candidates = []
82 end
83
84 def match
85   @candidates = []
86 end
87
88 def match
89   @candidates = []
90 end
91
92 def match
93   @candidates = []
94 end
95
96 def match
97   @candidates = []
98 end
99
100 def match
101   @candidates = []
102 end
```



2.2.3 Rootkits

Kernel Level

This is the most common type. They are known for their resistance to removal, since they run at the same privilege level as antivirus software.

Hypervisor Level

Modern processors have support for virtualization. Rootkits which use such processor specific technologies are called hypervisor rootkits, such as [blue pill](http://theinvisiblethings.blogspot.com/2006/06/introducing-blue-pill.html) and [SubVirt](https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/subvirt.pdf).

2.2.3 Rootkits

Firmware Level

Rootkits for firmware such as BIOS, ACPI tables, and device ROMS are known to exist. They have the highest chance of survival because currently no tools exist to verify/scan for firmware level rootkits.

2.2.4 Bootkits

Bootkits differ from rootkits in their installation process and how/when they take control of the operating system.

Bootkits start attacking the operating system before the operating system has started. They are able to completely violate the security of the target operating system.

2.2.5 Trojans

A **trojan** (or **trojan horse**) is a kind of malware that appears to the user to perform a legitimate function, but actually facilitates unauthorized access to the owner's machine.

An example of a trojan would be when you install a game you downloaded from the Internet onto your machine, but it might contain additional malicious code that is not part of the game. While you're playing the game, the secondary code would execute to perform its malicious activity.

2.2.6 Backdoors

A **backdoor** is software (or modification of software) which helps in bypassing authentication mechanisms. They can keep remote access open for later unauthorized purposes while trying to remain hidden.

For example, a backdoor in a login system might give you access when a specified username/password is entered, even though the credentials might not be a valid combination. **RATs (Remote Access Trojans)** are similar to backdoors.

2.2.7 Remote Access Trojans

A **Remote Access Trojan (RAT)** is a malicious remote administration tool, which an attacker uses to issue commands to the compromised host. A RAT uses a client-server model and has a user interface for easy administration.



2.2.8 Spyware

Spyware is software which spies on user activities to collect user information, such as what websites the user frequently visits, without the consent of the computer owner.

The information is sent to the author or owner of the spyware program after a certain amount of data has been collected. Normally, a system which has spyware also has other kinds of malware, such as rootkits or trojans, to hide their tracks and to keep control of the machine.

2.2.9 Botnets

Botnets are a collection of compromised computers which run commands automatically and autonomously, with the help of a command and control server. Botnets are typically created when a number of clients install the same malware. The hosts are usually infected via drive-by-downloads.

The controller or owner of the botnet is called the bot master and is usually the one who gives commands to the bots. Botnets are used by the bot master for reasons such as DDoS, sending spam, etc.

2.2.10 Ransomware

This type of malware encrypts files and demands the victim send money via Bitcoin, after which the user will be sent the key to unlock the files. The files are being held hostage until the victim pays the ransom, hence the term **ransomware**.

They are also called extortive malware, since they demand money in exchange for restoration of the victim's data.

2.2.11 Information Stealers

This type of malware basically steals data such as private encryption keys, login credentials, credit card data, competitor data (such as proprietary data, intellectual property, etc.), and other important data which could be used for many malicious reasons.



2.2.11 Information Stealers

Keyloggers

Keyloggers capture keystrokes as the victim is typing. This information is saved locally and later sent to the attacker.

Screen recorders

Screen recorders take screenshots of the active window on the victim's machine when a condition is met, such as a time interval. These images are saved locally and sent to the attacker as well.

2.2.11 Information Stealers

RAM scrapers

RAM scrapers attempt to steal information in memory while it's being processed. The reason for this is because in memory, everything is decrypted. This technique is well known for stealing credit card numbers in some big-name breaches within the last few years.

2.2 Malware Classifications

Basically, the point to note is that there is no clear line which distinguishes one form of malware from another. Normally, malware is found in pairs with multiple variants simultaneously active on the target system.

There are various other types of malware classifications, such as **Adware**, **Greyware**, **Scareware**, **Fakeware**, **PUPs** (Potentially Unwanted Programs), etc., but we will not be covering those types of malware as they're not what we'll be primarily hunting within our corporate environments.

2.2 Malware Classifications

Being familiar with malware and which classification it would fall under will help you understand the purpose of the malware and potentially what actions it took, or is going to take.

Next, we'll look at how the malware will reach the target system.

```
24 def initialize(experiment, observations, control, candidates):
25     """Initialize the experiment and observations"""
26     self.experiment = experiment
27     self.observations = observations
28     self.control = control
29     self.candidates = candidates
30     self.evaluate_candidates()
31
32     freeze
33
34     # TODO: the experiment's context
35     def context:
36         experiment.context
37
38     @property
39     def experiment_name:
40         experiment.name
41
42     def match(self, result):
43         """Match the result a match between the result and the control"""
44         return result == self.control
45
46     def match(self, result):
47         """Match the result a match between the result and the control"""
48         return result == self.control
49
50     def match(self, result):
51         """Match the result a match between the result and the control"""
52         return result == self.control
```

Malware Delivery



2.3 Malware Delivery

There are various ways malware can reach its target. Below are a few:

- Physical media
- Email (attachments)
- URL links
- Drive-by downloads
- Web advertising
- Social media
- File shares
- Software vulnerabilities

2.3.1 Physical Media

Malware that uses physical medium to spread generally doesn't have any other means to spread itself. Malware that uses a USB stick, for example, could infect the boot sector or be configured to autorun once inserted into the victim machine.

Perhaps the malware is not configured that way. Another method would be simply to put the malware into the USB stick and hope that your intended target(s) will run the malware.

2.3.1 Physical Media

Another method worth mentioning is **HIDs (Human Interface Device)**.

[USB Rubber Ducky](http://usbrubberducky.com/#!index.md), [USBdriveby](http://samy.pl/usbdiveby/), [Teensy](https://www.pjrc.com/teensy/), and [BadUSB](https://nakedsecurity.sophos.com/2014/10/06/badusb-now-with-do-it-yourself-instructions/) are examples of attacks where the attacker can create scripts to execute a set of commands, such as running malware, on a target system.

<http://usbrubberducky.com/#!index.md>

<http://samy.pl/usbdiveby/>

<https://www.pjrc.com/teensy/>

<https://nakedsecurity.sophos.com/2014/10/06/badusb-now-with-do-it-yourself-instructions/>

2.3.2 Email

This is one of the most common methods to attempt to infiltrate an organization. Due to users' poor security awareness, they are susceptible to falling victim to social engineering via phishing, spear phishing, and whaling attacks.

Why would an attacker spend hours, days, or even weeks surveying the network perimeter of the target in hopes of punching a hole through the firewall when a nicely crafted email, with a malicious attachment, will do the trick and get them inside? This attack vector is fruitful compared to finding a vulnerability through the firewall.

2.3.2 Email

Also, the email doesn't necessarily need to have an attachment.

The email can still be carefully crafted to lure the victim to click a link or even visit a website where the attacker hosts the malware.

2.3.3 URL Links

We have been reading about links thus far, and it should have its own section as this is an attack vector to deliver malware to the target.

With URL links, the link can direct to victim to a website where the malware is hosted, or the link could simply download the malware itself.

```
24 # Experiment class result is a dict
25 # observations - an array of Observations, in experiment
26 # control - the control observation
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   freeze
36
37   # Experiment context
38   experiment.context
39
40   # Experiment name
41   experiment.name
42
43   # ...
44
45   # ... test the result a match between ...
46   def match?
47     ...
48   end
49
50   # Experiment result
51   result = {}
52   result[:result] = 1.1
```



2.3.3 URL Links

As we have read already, links can be placed anywhere:

- Emails
- Documents (Word, Excel, etc.)
- Instant Messaging
- Social Media Feeds
- Etc.

[URL shortening](#) has also aided in making this a successful attack vector. The victim can't tell what they're clicking on by simply looking at the link.

2.3.4 Drive-by Downloads

A **drive-by download** site installs malware once a victim visits that site.

This means that the victim doesn't need to click on a link. Code in the web page, whether client-side or server-side, will attempt to install malware based on the victim's machine configuration (Windows 7 running an unpatched version of Flash for example). Typically, this is achieved by [exploit kits](https://blog.malwarebytes.com/threat-analysis/2017/03/exploit-kits-winter-2017-review/).

2.3.4 Drive-by Downloads

The attackers can purchase and build specific domains to spread malware via drive-by. Attackers can use popular news/events and use search engine optimization (SEO) techniques to push their malware-hosting website to the top of the search ranks to increase the chances of victims visiting the site.

Another technique can be used which is called a [watering hole attack](#).

2.3.5 Web Advertising

This technique is known as **malvertising**, delivering malware through online ads. This is achieved by buying ad space in popular, legitimate websites, but the ads have a malicious purpose.

The ads will either redirect the victim to a website hosting the malware or it will insert malware directly into the victim's machine. This can be done by clicking on the ad and even **without** clicking on the ad. You can read more [here](https://arstechnica.com/information-technology/2016/12/millions-exposed-to-malvertising-that-hid-attack-code-in-banner-pixels/).

2.3.5 Web Advertising

Another way to use ad-serving code is through browser extensions. In more recent news, a couple of Chrome extensions were targeted and displayed malicious ads.

You can read about this [here](https://www.forbes.com/sites/leemathews/2017/08/03/over-a-million-coders-targeted-by-chrome-extension-hack/#670c91059c9d).

2.3.6 Social Media

Social media continues to grow in popularity. The amount of users registered in various platforms are in the hundreds of millions. Even companies have social media accounts to promote their company and products.

Many of these social media platforms offer tools and features that will aid an attacker in their mission. They have the ability to post links in feeds and/or send direct messages to users.

```
24 # the experiment data result is a dict
25 # observations - an array of Observations, in experiment
26 # control - the control observation
27
28 def initialize(experiment, observations = [], control = null)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   freeze
36
37   @context =
38     experiment.context
39
40   @experiment_name =
41     experiment.name
42
43   @batch_size =
44     batch_size
45
46   @experiment/result_id = 1
```



2.3.6 Social Media

Within their attack campaign, the attackers will use some social engineering techniques, again, to convince the unsuspecting victims to click on a link or to download the malware.



2.3.7 Software Vulnerabilities

Software vulnerabilities refer to exploiting vulnerabilities within whitelisted applications running in the environment to inject executable code into the running application and take control of the process.

Attackers exploit the vulnerabilities (application bugs and/or flaws) with buffer overflows:

- Stack overflows
- Heap overflows

```
21 def initialize_experiment(experiment_name, observations = [], control = null)
22   # Create a new experiment
23   # @experiment - the experiment this result is for
24   # @observations - an array of Observations, or nil
25   # @control - the control observation
26   # @candidates - the candidates to evaluate
27
28   # Initialize the experiment
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations - [control]
33   evaluate_candidates
34
35   # Return the experiment
36   return self
37
38 def context
39   @context ||= ExperimentContext.new(self)
40 end
41
42 # Helper: the name of the experiment
43 def experiment_name
44   @experiment_name ||= experiment.name
45 end
46
47 # Helper: was the result a match between the control and the candidates?
48 def matched?
49   # ...
50   @experiment.result.sub 1.1
51 end
```

2.3.7 Software Vulnerabilities

Stack overflows

This is exploited by overflowing the buffers on the stack to get control of the flow of execution for the application to execute the malicious code.

Heap overflows

This is exploited by overwriting pointers in the heap (dynamically allocated memory space) to point to malicious code instead of its original location.

```
24
25
26 # EXPERIMENT: the experiment will result in 2
27 # observations - an array of Observations, in this case
28 # control - the control observation
29
30 def initialize(experiment, observations = [], control = nil)
31   @experiment = experiment
32   @observations = observations
33   @control = control
34   @candidates = observations + [control]
35   evaluate_candidates
36
37   freeze
38 end
39
40 # PUBLIC: the experiment's context
41 def context
42   @experiment.context
43 end
44
45 # PUBLIC: the experiment's name
46 def name
47   @experiment.name
48 end
49
50 # PUBLIC: the experiment's result
51 def result
52   @experiment.result
53 end
```


2.3 Malware Delivery

So far, we discussed how malware is classified and how it can reach a victim machines.

Next, we need to discuss how the malware attempts to remain undetected and dormant once it lands on the victim machine, as well as techniques to avoid analysis if found.

Malware Evasion Techniques



2.4 Malware Evasion Techniques

We'll begin this section with techniques on how malware attempts to run and [evade defenses](#).

Not only is the malware looking for a successful initial [execution](#), but it's also using techniques to [escalate privileges](#), [steal credentials](#), [exfiltrate data](#), and [maintain persistence](#), to name a few. Refer to the MITRE ATT&CK wiki [here](#) for more information.

https://attack.mitre.org/wiki/Defense_Evasion
<https://attack.mitre.org/wiki/Execution>
https://attack.mitre.org/wiki/Privilege_Escalation
https://attack.mitre.org/wiki/Credential_Access

<https://attack.mitre.org/wiki/Exfiltration>
<https://attack.mitre.org/wiki/Persistence>
https://attack.mitre.org/wiki/Technique_Matrix

2.4 Malware Evasion Techniques

Note that new methods and techniques are always being discovered by researchers and, most importantly, the adversary to remain covert and prevent detection.

As a hunter, you must always be looking into the latest techniques being shared and discovered.

2.4.1 Alternate Data Streams

Alternate Data Streams, or streams, are a feature of the NTFS file system. They are not available on FAT file systems or any other file system.

As per the Microsoft's [article](https://docs.microsoft.com/en-us/archive/blogs/askcore/alternate-data-streams-in-ntfs), the original data stream is the file data itself, which is the data stream with no name. All other streams have a name. Alternate Data Streams can be used to store file meta data and any other type of data.

2.4.1 Alternate Data Streams

To explain this concept, we can type the following into the command prompt:

```
echo "This is not ADS" > file.txt
```

```
echo "This is in ADS" > file.txt:stream1
```

We can also create streams programmatically. In the [CreateFile](#) Windows API, just append “:stream_name” to the file name, where “stream_name” is the name of the data stream. We could also use the [WriteFile](#) Windows API function to write the data.

2.4.1 Alternate Data Streams

Example program that writes to Alternate Data Streams is presented on the right.

```
#include <windows.h>
#include <stdio.h>

void main() {
    ...
    hStream = CreateFile("file.txt:stream2",
                        GENERIC_WRITE,
                        FILE_SHARE_WRITE,
                        NULL,
                        OPEN_ALWAYS,
                        0,
                        NULL);
    if(hStream == INVALID_HANDLE_VALUE)
        printf("Cannot open file.txt:stream2\n");
    else
        WriteFile(hStream, "This data is hidden in the
stream. Can you read it???", 53, &dwRet, NULL);
}
```

2.4.1 Alternate Data Streams

Let's use the DIR command to attempt to see the regular txt file and the ADS.

```
C:\Users\elshunter\Desktop>dir
Volume in drive C has no label.
Volume Serial Number is 4247-60E2

Directory of C:\Users\elshunter\Desktop

08/04/2017  01:22 PM    <DIR>          .
08/04/2017  01:22 PM    <DIR>          ..
08/04/2017  01:22 PM                20 file.txt
                    1 File(s)                20 bytes
                    2 Dir(s)      7,552,462,848 bytes free

C:\Users\elshunter\Desktop>dir /r
Volume in drive C has no label.
Volume Serial Number is 4247-60E2

Directory of C:\Users\elshunter\Desktop

08/04/2017  01:22 PM    <DIR>          .
08/04/2017  01:22 PM    <DIR>          ..
08/04/2017  01:22 PM                20 file.txt
                                19 file.txt:stream1:$DATA
                    1 File(s)                20 bytes
                    2 Dir(s)      7,552,462,848 bytes free
```

2.4.1 Alternate Data Streams

Here, we attempt to view the contents of the txt file and ADS using **TYPE** and **MORE**.

```
C:\Users\elshunter\Desktop>type file.txt
"This is not ADS"

C:\Users\elshunter\Desktop>type file.txt:stream1
The filename, directory name, or volume label syntax is incorrect.

C:\Users\elshunter\Desktop>more < file.txt
"This is not ADS"

C:\Users\elshunter\Desktop>more < file.txt:stream1
"This is in ADS"
```

2.4.1 Alternate Data Streams

Another tool to view alternate data streams is the [Streams](#) tool bundled within [Sysinternals](#).

streams.exe c:\users\elshunter\desktop\file.txt

```
c:\Users\elshunter\Documents\Tools\SysinternalsSuite>streams.exe c:\Users\elshunter\Desktop\file.txt

streams v1.60 - Reveal NTFS alternate streams.
Copyright (C) 2005-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

c:\Users\elshunter\Desktop\file.txt:
:stream1:$DATA 19
```

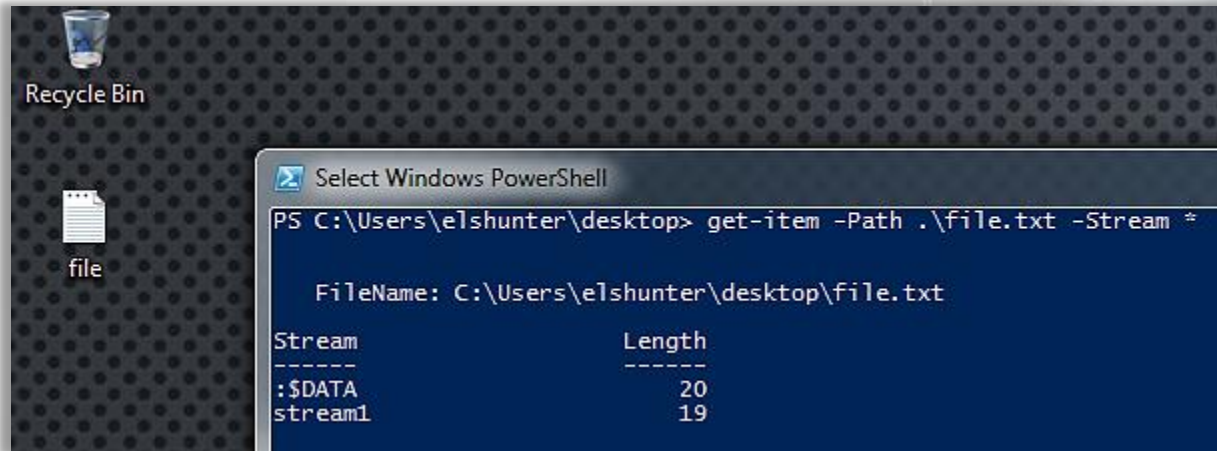
2.4.1 Alternate Data Streams

This tool was useful prior to the Windows PowerShell days. PowerShell's cmdlet [Get-Item](#) has the capability to retrieve alternate data stream information.

We'll need to use the “**-Stream**” parameter in order to do so.

2.4.1 Alternate Data Streams

`get-item -path .\file.txt -stream *`



2.4.1 Alternate Data Streams

Note that Microsoft uses ADS for non-nefarious reasons. For instance, via the **Zone.Identifier (Zone 3) ADS**, we can tell if a file or binary was downloaded from the Internet (**Internet Zone**).

Value	Zone
0	My Computer
1	Local Intranet Zone
2	Trusted Sites Zone
3	Internet Zone
4	Restricted Sites Zone

2.4.1 Alternate Data Streams

To retrieve this information, we need to use the [Get-Content](#) cmdlet within PowerShell.

get-item -path .\putty-64bit-0.70-installer.msi -stream *

```
PS C:\Users\elshunter\Downloads> get-item -path .\putty-64bit-0.70-installer.msi -stream *
```

Stream	Length
:\$DATA	3048960
Zone.Identifier	26

```
PS C:\Users\elshunter\Downloads> get-content -path .\putty-64bit-0.70-installer.msi -stream zone.identifier  
[ZoneTransfer]  
ZoneId=3
```

2.4.1 Alternate Data Streams

Within [PowerShell version 5](#) and newer versions, there is a lot more information that is displayed when using the **Get-Item** cmdlet and the “**-Stream**” parameter.

2.4.1 Alternate Data Streams

Here you can see the Get-Item command being used with the -Stream option.

```
PS C:\Users\██████\Downloads> Get-Item -Path .\NwInvestigatorSetup-10.6.1.1.696.x64.msi -Stream *
```

PSPath	: Microsoft.PowerShell.Core\FileSystem::C:\Users\██████\Downloads\NwInvestigatorSetup-10.6.1.1.696.x64.msi
PSParentPath	: ::\$DATA
PSChildName	: NwInvestigatorSetup-10.6.1.1.696.x64.msi::\$DATA
PSDrive	: C
PSProvider	: Microsoft.PowerShell.Core\FileSystem
PSIsContainer	: False
FileName	: C:\Users\██████\Downloads\NwInvestigatorSetup-10.6.1.1.696.x64.msi
Stream	: ::\$DATA
Length	: 74014720

PSPath	: Microsoft.PowerShell.Core\FileSystem::C:\Users\██████\Downloads\NwInvestigatorSetup-10.6.1.1.696.x64.msi
PSParentPath	: :Zone.Identifier
PSChildName	: NwInvestigatorSetup-10.6.1.1.696.x64.msi:Zone.Identifier
PSDrive	: C
PSProvider	: Microsoft.PowerShell.Core\FileSystem
PSIsContainer	: False
FileName	: C:\Users\██████\Downloads\NwInvestigatorSetup-10.6.1.1.696.x64.msi
Stream	: Zone.Identifier
Length	: 26

```
PS C:\Users\██████\Downloads> get-content -path .\NwInvestigatorSetup-10.6.1.1.696.x64.msi -stream zone.identifier  
[ZoneTransfer]  
ZoneId=3
```

2.4.1 Alternate Data Streams

An example of malware that attempts to remove clues that it was downloaded from the Internet by deleting the Zone.Identifier from the file is [CoinMiner](https://secreary.com/ReversingMalware/CoinMiner/).

Deletes Zone.Identifier of the file:

```
004025D6 lea     edx, [esp+25A0h+FileName]
004025DD lea     ecx, [esp+25A0h+Filename] ; lpFileName
004025E4 call    copy_via_mapping
004025E9 mov     edx, offset aZone_identifie ; ":Zone.Identifier"
004025EE lea     ecx, [esp+25A0h+FileName] ; lpString2
004025F5 call    sub_401040 ; ???
004025FA mov     esi, eax
004025FC push    esi ; C:\Users\John Doe\AppData\Local\hjfdWLFJw\taskman.exe:Zone.Identifier
004025FD call    ds:DeleteFileW
00402603 push    esi ; void *
00402604 call    j__free
00402609 add     esp, 4
```

Credit: <https://secreary.com/ReversingMalware/CoinMiner/>

2.4.2 Injections

In the upcoming slides, we'll look at various injection techniques used by malware to inject itself into other processes, threads, etc.

2.4.2.1 DLL Injections

2. Open Process

Once the malware finds the process, it opens the process.

- Windows API: [GetModuleHandle\(\)](#), [GetProcAddress\(\)](#), [OpenProcess\(\)](#)

3. Allocate Memory

The malware then needs to find a location in order to write the path to the malicious DLL.

- Windows API: [VirtualAllocEx\(\)](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms683199\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms683199(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms683212\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms683212(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684320\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684320(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890(v=vs.85).aspx)

```
21 # @param: context
22 # @param: experiment - the experiment this result is from
23 # @param: observations - an array of Observations, in ascending order
24 # @param: control - the control observation
25
26 def initialize(experiment, observations = [], control = null)
27   @experiment = experiment
28   @observations = observations
29   @control = control
30   @candidates = observations - @control
31 end
32
33 # @param: the experiment's context
34 def context
35   @experiment.context
36 end
37
38 # @param: the name of the experiment
39 def experiment_name
40   @experiment.name
41 end
42
43 # @param: the result a match between all candidates
44 def match
45   @match
46 end
47
48 # @param: result
49 def result
50   @result
51 end
```

2.4.2.1 DLL Injections

4. Copy

The malware will write the path to the malicious DLL into the allocated memory location.

- Window API: [WriteProcessMemory\(\)](#)

5. Execute

The malware will execute the malicious DLL in another process by starting a new thread.

- Window API: [CreateRemoteThread\(\)](#), [LoadLibrary\(\)](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682437\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682437(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684175\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684175(v=vs.85).aspx)

```
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

2.4.2.1 DLL Injections

It's important to note that the **documented** Windows API **CreateRemoteThread()** function is not the only function that can be used. There are **undocumented** functions that can be used as well.

2.4.2.1 DLL Injections

NTCreateThreadEx()

The steps to utilize this undocumented function are a bit more involved. This function needs to be loaded from Ntdll.dll.

RtlCreateUserThread()

Both **Mimikatz** and **Metasploit** use this undocumented function. This function also needs to be loaded from Ntdll.dll.

<https://undocumented.ntinternals.net/index.html?page=UserMode/Undocumented%20Functions/NT%20Objects/Thread/NtCreateThread.html>

<https://undocumented.ntinternals.net/index.html?page=UserMode/Undocumented%20Functions/Executable%20Images/RtlCreateUserThread.html>

2.4.2.2 Reflective DLL Injection

This technique was discovered and shared by Steven Fewer. As stated on the GitHub [page](#):

“**Reflective DLL injection** is a library injection technique in which the concept of reflective programming is employed to perform the loading of a library from **memory** into a host process.”

2.4.2.2 Reflective DLL Injection

Frameworks, such as **Metasploit** and **PowerShell Empire**, have this capability incorporated into them.

APTs and malware authors, especially advanced ones, will most likely not use these frameworks, but instead will create their own tools using C/C++.

2.4.2.3 Thread Hijacking

With this technique, the malware will not need to create a new process or thread. Instead, it will loop through threads of a process on the target system and perform similar steps of **DLL Injection** but with slight differences.

1. Locate Thread

Malware needs to find a target thread to inject into.

- Windows API: [CreateToolhelp32Snapshot\(\)](#), [Thread32First\(\)](#), [Thread32Next\(\)](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms686728\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686728(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms686731\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686731(v=vs.85).aspx)

```
21 def __init__(self):
22     self.experiment = Experiment()
23     self.observations = []
24     self.control = None
25     self.candidates = []
26     self.evaluate_candidates()
27
28 def initialize(experiment, observations = [], control = None):
29     @experiment = experiment
30     @observations = observations
31     @control = control
32     @candidates = observations + [control]
33     evaluate_candidates
34
35 freeze
36
37 def __str__(self):
38     return f"Experiment: {self.experiment.name}, Observations: {self.observations}, Control: {self.control}, Candidates: {self.candidates}"
39
40 def __repr__(self):
41     return f"Experiment: {self.experiment.name}, Observations: {self.observations}, Control: {self.control}, Candidates: {self.candidates}"
42
43 def __eq__(self, other):
44     return self.experiment.name == other.experiment.name and self.observations == other.observations and self.control == other.control and self.candidates == other.candidates
45
46 def __neq__(self, other):
47     return not self.__eq__(other)
48
49 def __hash__(self):
50     return hash(self.experiment.name + str(self.observations) + str(self.control) + str(self.candidates))
51
52 def __getitem__(self, index):
53     return self.observations[index]
54
55 def __setitem__(self, index, value):
56     self.observations[index] = value
57
58 def __len__(self):
59     return len(self.observations)
60
61 def __iter__(self):
62     return iter(self.observations)
63
64 def __contains__(self, value):
65     return value in self.observations
66
67 def __del__(self):
68     pass
```

2.4.2.3 Thread Hijacking

2. Open Thread

The malware opens the thread.

- Windows API: OpenThread()

3. Suspend Thread

The thread needs to be suspended in order to inject.

- Windows API: SuspendThread()

2.4.2.3 Thread Hijacking

4. Allocate Memory

The malware then needs to find a location in order to write the path to the malicious DLL, or shellcode.

- Windows API: [VirtualAllocEx\(\)](#)

5. Copy

The malware will write the path to the malicious DLL, shellcode, or address to LoadLibrary into the allocated memory location.

- Window API: [WriteProcessMemory\(\)](#)

```
19 def initialize_experiment(experiment, observations = [], control = null)
20   # Initialize the experiment
21   # @experiment: the experiment to be initialized
22   # @observations: an array of Observations, or null
23   # @control: the control observation
24   # @candidates: an array of candidates, or null
25   # @initialize_candidates: a function to initialize candidates
26   # @initialize_control: a function to initialize control
27   # @initialize_observations: a function to initialize observations
28   # @initialize_experiment: a function to initialize the experiment
29   # @initialize_experiment: a function to initialize the experiment
30   # @initialize_experiment: a function to initialize the experiment
31   # @initialize_experiment: a function to initialize the experiment
32   # @initialize_experiment: a function to initialize the experiment
33   # @initialize_experiment: a function to initialize the experiment
34   # @initialize_experiment: a function to initialize the experiment
35   # @initialize_experiment: a function to initialize the experiment
36   # @initialize_experiment: a function to initialize the experiment
37   # @initialize_experiment: a function to initialize the experiment
38   # @initialize_experiment: a function to initialize the experiment
39   # @initialize_experiment: a function to initialize the experiment
40   # @initialize_experiment: a function to initialize the experiment
41   # @initialize_experiment: a function to initialize the experiment
42   # @initialize_experiment: a function to initialize the experiment
43   # @initialize_experiment: a function to initialize the experiment
44   # @initialize_experiment: a function to initialize the experiment
45   # @initialize_experiment: a function to initialize the experiment
46   # @initialize_experiment: a function to initialize the experiment
47   # @initialize_experiment: a function to initialize the experiment
48   # @initialize_experiment: a function to initialize the experiment
49   # @initialize_experiment: a function to initialize the experiment
50   # @initialize_experiment: a function to initialize the experiment
51   # @initialize_experiment: a function to initialize the experiment
52   # @initialize_experiment: a function to initialize the experiment
53   # @initialize_experiment: a function to initialize the experiment
54   # @initialize_experiment: a function to initialize the experiment
55   # @initialize_experiment: a function to initialize the experiment
56   # @initialize_experiment: a function to initialize the experiment
57   # @initialize_experiment: a function to initialize the experiment
58   # @initialize_experiment: a function to initialize the experiment
59   # @initialize_experiment: a function to initialize the experiment
60   # @initialize_experiment: a function to initialize the experiment
61   # @initialize_experiment: a function to initialize the experiment
62   # @initialize_experiment: a function to initialize the experiment
63   # @initialize_experiment: a function to initialize the experiment
64   # @initialize_experiment: a function to initialize the experiment
65   # @initialize_experiment: a function to initialize the experiment
66   # @initialize_experiment: a function to initialize the experiment
67   # @initialize_experiment: a function to initialize the experiment
68   # @initialize_experiment: a function to initialize the experiment
69   # @initialize_experiment: a function to initialize the experiment
70   # @initialize_experiment: a function to initialize the experiment
71   # @initialize_experiment: a function to initialize the experiment
72   # @initialize_experiment: a function to initialize the experiment
73   # @initialize_experiment: a function to initialize the experiment
74   # @initialize_experiment: a function to initialize the experiment
75   # @initialize_experiment: a function to initialize the experiment
76   # @initialize_experiment: a function to initialize the experiment
77   # @initialize_experiment: a function to initialize the experiment
78   # @initialize_experiment: a function to initialize the experiment
79   # @initialize_experiment: a function to initialize the experiment
80   # @initialize_experiment: a function to initialize the experiment
81   # @initialize_experiment: a function to initialize the experiment
82   # @initialize_experiment: a function to initialize the experiment
83   # @initialize_experiment: a function to initialize the experiment
84   # @initialize_experiment: a function to initialize the experiment
85   # @initialize_experiment: a function to initialize the experiment
86   # @initialize_experiment: a function to initialize the experiment
87   # @initialize_experiment: a function to initialize the experiment
88   # @initialize_experiment: a function to initialize the experiment
89   # @initialize_experiment: a function to initialize the experiment
90   # @initialize_experiment: a function to initialize the experiment
91   # @initialize_experiment: a function to initialize the experiment
92   # @initialize_experiment: a function to initialize the experiment
93   # @initialize_experiment: a function to initialize the experiment
94   # @initialize_experiment: a function to initialize the experiment
95   # @initialize_experiment: a function to initialize the experiment
96   # @initialize_experiment: a function to initialize the experiment
97   # @initialize_experiment: a function to initialize the experiment
98   # @initialize_experiment: a function to initialize the experiment
99   # @initialize_experiment: a function to initialize the experiment
100  # @initialize_experiment: a function to initialize the experiment
```

2.4.2.3 Thread Hijacking

6. Resume Thread

Once the malware has performed its injection, the thread resumes.

- Windows API: [ResumeThread\(\)](#)

```
def initialize_experiment(experiment, observations = [], control = null)
  # Create the experiment's context
  experiment.context = Context.new
  # Create the experiment's observations
  experiment.observations = observations
  # Create the experiment's control
  experiment.control = control
  # Initialize the experiment's state
  experiment.initialize
end

def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @state = :initial
  @freeze = false
end

def freeze
  @freeze = true
end

def unfreeze
  @freeze = false
end

def run(experiment)
  # Run the experiment's context
  experiment.context.run
end

def run(experiment, observations = [], control = null)
  # Run the experiment's observations
  experiment.observations.run
end

def run(experiment, observations = [], control = null)
  # Run the experiment's control
  experiment.control.run
end

def run(experiment, observations = [], control = null)
  # Run the experiment's state
  experiment.state.run
end

def run(experiment, observations = [], control = null)
  # Run the experiment's result
  experiment.result.run
end
```

2.4.2.4 PE Injection

PE (Portable Executable) Injection is similar to **DLL Injection**, but one notable difference is that, just like **Reflective DLL Injection**, the malicious DLL doesn't have to reside on disk.

WriteProcessMemory() will be used by this technique, but not to write the path to the malicious DLL. Instead, it will be used to write the malicious code into that location. There is no need to use **LoadLibrary()** either with this technique.

2.4.2.4 PE Injection

Since the PE is injected into another process, it will have a new base address. The base address is the starting address of a memory-mapped EXE or DLL. The malware will need to loop through its relocation descriptors to find its addresses. You can read more about PE Headers in the document titled [Peering Inside the PE: A Tour of the Win32 Portable Executable File Format](https://msdn.microsoft.com/en-us/library/ms809762.aspx).

2.4.2.5 Process Hollowing

Process Hollowing is the technique where malware will unmap the legitimate code from memory of the process and overwrite memory of the process with a malicious binary.

1. Create Process

Malware will create a new process in a suspended state to host the malicious code. The primary thread will also be in a suspended state.

- Window API: [CreateProcess\(\)](#)

2.4.2.5 Process Hollowing

2. Un-map Memory

The malware at this point will un-map the legitimate code from memory.

- Windows API: [ZwUnmapViewOfSection\(\)](#), [NtUnmapViewOfSection\(\)](#)

3. Allocate & Write

Same as DLL Injection technique, the malware will allocate memory locations for malicious code and write each section of the malware into this space.

- Windows API: [VirtualAllocEx\(\)](#), [WriteProcessMemory\(\)](#)

[https://msdn.microsoft.com/ru-ru/library/ff567119\(v=vs.85\).aspx](https://msdn.microsoft.com/ru-ru/library/ff567119(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/ff557711\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/ff557711(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx)

2.4.2.5 Process Hollowing

3. Set Entry Point

The malware will set the entry point to new code section.

- Windows API: [SetThreadContext\(\)](#)

4. Resume

The malware will take the process out of suspended state.

- Windows API: [ResumeThread\(\)](#)

```
34 def initialize_experiment(self, experiment_name, control):
35     """Initialize the experiment with results in 2D
36     @observations: an array of Observations, in 2D
37     @control: the control observation
38     """
39     self.initialize(experiment, observations = [], control = control)
40     @experiment = experiment
41     @observations = observations
42     @control = control
43     @candidates = observations + [control]
44     evaluate_candidates
45
46     freeze
47
48     # TODO: the experiment's control
49     self.control = control
50
51     # TODO: the name of the experiment
52     self.experiment_name = experiment_name
53
54     # TODO: the results a match between all
55     self.matched = False
56
57     # TODO: the results a match between all
58     self.matched = False
59
60     # TODO: the results a match between all
61     self.matched = False
```

2.4.2.6 Hook Injection

Malware uses **hooking** to intercept events, and based on a particular triggered event, the malware will respond as instructed.

This is accomplished using the Windows API [SetWindowsHookEx\(\)](#).

```
def initialize(experiment, observations = {}, control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @candidates = observations - {control}
  evaluate_candidates

  freeze
end

def initialize_experiment's context
  t.context

  # Initialize the name of the experiment
  def experiment_name
    experiment.name
  end

  # A helper method to check the results a match between all
  def matched?
    # ...
  end
end
```

2.4.2.6 Hook Injection

This can be used to monitor the keyboard (**WH_KEYBOARD**) and mouse (**WH_MOUSE**) input. WH_KEYBOARD and WH_MOUSE are hook types. There are different hook types that can be passed as an argument to SetWindowsHookEx(). You can read more about hook types [here](#). Malware can also use this technique to load a malicious DLL based on a specific event.

2.4.2.7 KernelMode Rootkits: SSDT Hooks

SSDT stands for System Service Descriptor Table. The Windows Kernel uses this table to lookup system functions and each table entry points to function code. Every SSDT entry will point to the system kernel (**ntoskrnl.exe**) or the GUI driver (**win32k.sys**).

2.4.2.7 KernelMode Rootkits: SSDT Hooks

For each entry in the SSDT table, there is a suitable function in kernel mode which completes the task specified by Native API. The SSDT resides in the kernel and is exported as **KeServiceDescriptorTable()**.

The representation can be pictured as:



```
21 # KernelMode Rootkit
22
23 # Experiment - the experiment this rootkit is for
24 # observations - an array of Observations, in memory
25 # control - the control Observation
26
27 def initialize(experiment, observations = [], control = null)
28   @experiment = experiment
29   @observations = observations
30   @control = control
31   @candidates = observations - @control
32   evaluate_candidates
33
34   freeze
35 end
36
37 # MAIN: the experiment's context
38 def context
39   @experiment.context
40 end
41
42 end
```

2.4.2.7 KernelMode Rootkits: SSDT Hooks

With SSDT Hooking, it will modify the pointers within this table to point to a location that the rootkit controls, such as a malicious function.

The effects of this technique are global to the system.

```
18 def initialize(experiment, observations = 100, candidates = 100)
19   @experiment = experiment
20   @observations = observations
21   @control = control
22   @candidates = observations - 1
23   evaluate_candidates
24
25   freeze
26 end
27
28 # PARS: the experiment's context
29 def context
30   experiment.context
31 end
32
33 # PARS: the experiment's name
34 def experiment_name
35   experiment.name
36 end
37
38 # PARS: the result's match
39 def match?
40   result.match?
41 end
42
43 # PARS: the result's match
44 def result
45   result
46 end
47
48 # PARS: the result's match
49 def result
50   result
51 end
52
53 # PARS: the result's match
54 def result
55   result
56 end
57
58 # PARS: the result's match
59 def result
60   result
61 end
62
63 # PARS: the result's match
64 def result
65   result
66 end
67
68 # PARS: the result's match
69 def result
70   result
71 end
72
73 # PARS: the result's match
74 def result
75   result
76 end
77
78 # PARS: the result's match
79 def result
80   result
81 end
82
83 # PARS: the result's match
84 def result
85   result
86 end
87
88 # PARS: the result's match
89 def result
90   result
91 end
92
93 # PARS: the result's match
94 def result
95   result
96 end
97
98 # PARS: the result's match
99 def result
100   result
101 end
102
103 # PARS: the result's match
104 def result
105   result
106 end
107
108 # PARS: the result's match
109 def result
110   result
111 end
112
113 # PARS: the result's match
114 def result
115   result
116 end
117
118 # PARS: the result's match
119 def result
120   result
121 end
122
123 # PARS: the result's match
124 def result
125   result
126 end
127
128 # PARS: the result's match
129 def result
130   result
131 end
132
133 # PARS: the result's match
134 def result
135   result
136 end
137
138 # PARS: the result's match
139 def result
140   result
141 end
142
143 # PARS: the result's match
144 def result
145   result
146 end
147
148 # PARS: the result's match
149 def result
150   result
151 end
152
153 # PARS: the result's match
154 def result
155   result
156 end
157
158 # PARS: the result's match
159 def result
160   result
161 end
162
163 # PARS: the result's match
164 def result
165   result
166 end
167
168 # PARS: the result's match
169 def result
170   result
171 end
172
173 # PARS: the result's match
174 def result
175   result
176 end
177
178 # PARS: the result's match
179 def result
180   result
181 end
182
183 # PARS: the result's match
184 def result
185   result
186 end
187
188 # PARS: the result's match
189 def result
190   result
191 end
192
193 # PARS: the result's match
194 def result
195   result
196 end
197
198 # PARS: the result's match
199 def result
200   result
201 end
202
203 # PARS: the result's match
204 def result
205   result
206 end
207
208 # PARS: the result's match
209 def result
210   result
211 end
212
213 # PARS: the result's match
214 def result
215   result
216 end
217
218 # PARS: the result's match
219 def result
220   result
221 end
222
223 # PARS: the result's match
224 def result
225   result
226 end
227
228 # PARS: the result's match
229 def result
230   result
231 end
232
233 # PARS: the result's match
234 def result
235   result
236 end
237
238 # PARS: the result's match
239 def result
240   result
241 end
242
243 # PARS: the result's match
244 def result
245   result
246 end
247
248 # PARS: the result's match
249 def result
250   result
251 end
252
253 # PARS: the result's match
254 def result
255   result
256 end
257
258 # PARS: the result's match
259 def result
260   result
261 end
262
263 # PARS: the result's match
264 def result
265   result
266 end
267
268 # PARS: the result's match
269 def result
270   result
271 end
272
273 # PARS: the result's match
274 def result
275   result
276 end
277
278 # PARS: the result's match
279 def result
280   result
281 end
282
283 # PARS: the result's match
284 def result
285   result
286 end
287
288 # PARS: the result's match
289 def result
290   result
291 end
292
293 # PARS: the result's match
294 def result
295   result
296 end
297
298 # PARS: the result's match
299 def result
300   result
301 end
302
303 # PARS: the result's match
304 def result
305   result
306 end
307
308 # PARS: the result's match
309 def result
310   result
311 end
312
313 # PARS: the result's match
314 def result
315   result
316 end
317
318 # PARS: the result's match
319 def result
320   result
321 end
322
323 # PARS: the result's match
324 def result
325   result
326 end
327
328 # PARS: the result's match
329 def result
330   result
331 end
332
333 # PARS: the result's match
334 def result
335   result
336 end
337
338 # PARS: the result's match
339 def result
340   result
341 end
342
343 # PARS: the result's match
344 def result
345   result
346 end
347
348 # PARS: the result's match
349 def result
350   result
351 end
352
353 # PARS: the result's match
354 def result
355   result
356 end
357
358 # PARS: the result's match
359 def result
360   result
361 end
362
363 # PARS: the result's match
364 def result
365   result
366 end
367
368 # PARS: the result's match
369 def result
370   result
371 end
372
373 # PARS: the result's match
374 def result
375   result
376 end
377
378 # PARS: the result's match
379 def result
380   result
381 end
382
383 # PARS: the result's match
384 def result
385   result
386 end
387
388 # PARS: the result's match
389 def result
390   result
391 end
392
393 # PARS: the result's match
394 def result
395   result
396 end
397
398 # PARS: the result's match
399 def result
400   result
401 end
402
403 # PARS: the result's match
404 def result
405   result
406 end
407
408 # PARS: the result's match
409 def result
410   result
411 end
412
413 # PARS: the result's match
414 def result
415   result
416 end
417
418 # PARS: the result's match
419 def result
420   result
421 end
422
423 # PARS: the result's match
424 def result
425   result
426 end
427
428 # PARS: the result's match
429 def result
430   result
431 end
432
433 # PARS: the result's match
434 def result
435   result
436 end
437
438 # PARS: the result's match
439 def result
440   result
441 end
442
443 # PARS: the result's match
444 def result
445   result
446 end
447
448 # PARS: the result's match
449 def result
450   result
451 end
452
453 # PARS: the result's match
454 def result
455   result
456 end
457
458 # PARS: the result's match
459 def result
460   result
461 end
462
463 # PARS: the result's match
464 def result
465   result
466 end
467
468 # PARS: the result's match
469 def result
470   result
471 end
472
473 # PARS: the result's match
474 def result
475   result
476 end
477
478 # PARS: the result's match
479 def result
480   result
481 end
482
483 # PARS: the result's match
484 def result
485   result
486 end
487
488 # PARS: the result's match
489 def result
490   result
491 end
492
493 # PARS: the result's match
494 def result
495   result
496 end
497
498 # PARS: the result's match
499 def result
500   result
501 end
502
503 # PARS: the result's match
504 def result
505   result
506 end
507
508 # PARS: the result's match
509 def result
510   result
511 end
512
513 # PARS: the result's match
514 def result
515   result
516 end
517
518 # PARS: the result's match
519 def result
520   result
521 end
522
523 # PARS: the result's match
524 def result
525   result
526 end
527
528 # PARS: the result's match
529 def result
530   result
531 end
532
533 # PARS: the result's match
534 def result
535   result
536 end
537
538 # PARS: the result's match
539 def result
540   result
541 end
542
543 # PARS: the result's match
544 def result
545   result
546 end
547
548 # PARS: the result's match
549 def result
550   result
551 end
552
553 # PARS: the result's match
554 def result
555   result
556 end
557
558 # PARS: the result's match
559 def result
560   result
561 end
562
563 # PARS: the result's match
564 def result
565   result
566 end
567
568 # PARS: the result's match
569 def result
570   result
571 end
572
573 # PARS: the result's match
574 def result
575   result
576 end
577
578 # PARS: the result's match
579 def result
580   result
581 end
582
583 # PARS: the result's match
584 def result
585   result
586 end
587
588 # PARS: the result's match
589 def result
590   result
591 end
592
593 # PARS: the result's match
594 def result
595   result
596 end
597
598 # PARS: the result's match
599 def result
600   result
601 end
602
603 # PARS: the result's match
604 def result
605   result
606 end
607
608 # PARS: the result's match
609 def result
610   result
611 end
612
613 # PARS: the result's match
614 def result
615   result
616 end
617
618 # PARS: the result's match
619 def result
620   result
621 end
622
623 # PARS: the result's match
624 def result
625   result
626 end
627
628 # PARS: the result's match
629 def result
630   result
631 end
632
633 # PARS: the result's match
634 def result
635   result
636 end
637
638 # PARS: the result's match
639 def result
640   result
641 end
642
643 # PARS: the result's match
644 def result
645   result
646 end
647
648 # PARS: the result's match
649 def result
650   result
651 end
652
653 # PARS: the result's match
654 def result
655   result
656 end
657
658 # PARS: the result's match
659 def result
660   result
661 end
662
663 # PARS: the result's match
664 def result
665   result
666 end
667
668 # PARS: the result's match
669 def result
670   result
671 end
672
673 # PARS: the result's match
674 def result
675   result
676 end
677
678 # PARS: the result's match
679 def result
680   result
681 end
682
683 # PARS: the result's match
684 def result
685   result
686 end
687
688 # PARS: the result's match
689 def result
690   result
691 end
692
693 # PARS: the result's match
694 def result
695   result
696 end
697
698 # PARS: the result's match
699 def result
700   result
701 end
702
703 # PARS: the result's match
704 def result
705   result
706 end
707
708 # PARS: the result's match
709 def result
710   result
711 end
712
713 # PARS: the result's match
714 def result
715   result
716 end
717
718 # PARS: the result's match
719 def result
720   result
721 end
722
723 # PARS: the result's match
724 def result
725   result
726 end
727
728 # PARS: the result's match
729 def result
730   result
731 end
732
733 # PARS: the result's match
734 def result
735   result
736 end
737
738 # PARS: the result's match
739 def result
740   result
741 end
742
743 # PARS: the result's match
744 def result
745   result
746 end
747
748 # PARS: the result's match
749 def result
750   result
751 end
752
753 # PARS: the result's match
754 def result
755   result
756 end
757
758 # PARS: the result's match
759 def result
760   result
761 end
762
763 # PARS: the result's match
764 def result
765   result
766 end
767
768 # PARS: the result's match
769 def result
770   result
771 end
772
773 # PARS: the result's match
774 def result
775   result
776 end
777
778 # PARS: the result's match
779 def result
780   result
781 end
782
783 # PARS: the result's match
784 def result
785   result
786 end
787
788 # PARS: the result's match
789 def result
790   result
791 end
792
793 # PARS: the result's match
794 def result
795   result
796 end
797
798 # PARS: the result's match
799 def result
800   result
801 end
802
803 # PARS: the result's match
804 def result
805   result
806 end
807
808 # PARS: the result's match
809 def result
810   result
811 end
812
813 # PARS: the result's match
814 def result
815   result
816 end
817
818 # PARS: the result's match
819 def result
820   result
821 end
822
823 # PARS: the result's match
824 def result
825   result
826 end
827
828 # PARS: the result's match
829 def result
830   result
831 end
832
833 # PARS: the result's match
834 def result
835   result
836 end
837
838 # PARS: the result's match
839 def result
840   result
841 end
842
843 # PARS: the result's match
844 def result
845   result
846 end
847
848 # PARS: the result's match
849 def result
850   result
851 end
852
853 # PARS: the result's match
854 def result
855   result
856 end
857
858 # PARS: the result's match
859 def result
860   result
861 end
862
863 # PARS: the result's match
864 def result
865   result
866 end
867
868 # PARS: the result's match
869 def result
870   result
871 end
872
873 # PARS: the result's match
874 def result
875   result
876 end
877
878 # PARS: the result's match
879 def result
880   result
881 end
882
883 # PARS: the result's match
884 def result
885   result
886 end
887
888 # PARS: the result's match
889 def result
890   result
891 end
892
893 # PARS: the result's match
894 def result
895   result
896 end
897
898 # PARS: the result's match
899 def result
900   result
901 end
902
903 # PARS: the result's match
904 def result
905   result
906 end
907
908 # PARS: the result's match
909 def result
910   result
911 end
912
913 # PARS: the result's match
914 def result
915   result
916 end
917
918 # PARS: the result's match
919 def result
920   result
921 end
922
923 # PARS: the result's match
924 def result
925   result
926 end
927
928 # PARS: the result's match
929 def result
930   result
931 end
932
933 # PARS: the result's match
934 def result
935   result
936 end
937
938 # PARS: the result's match
939 def result
940   result
941 end
942
943 # PARS: the result's match
944 def result
945   result
946 end
947
948 # PARS: the result's match
949 def result
950   result
951 end
952
953 # PARS: the result's match
954 def result
955   result
956 end
957
958 # PARS: the result's match
959 def result
960   result
961 end
962
963 # PARS: the result's match
964 def result
965   result
966 end
967
968 # PARS: the result's match
969 def result
970   result
971 end
972
973 # PARS: the result's match
974 def result
975   result
976 end
977
978 # PARS: the result's match
979 def result
980   result
981 end
982
983 # PARS: the result's match
984 def result
985   result
986 end
987
988 # PARS: the result's match
989 def result
990   result
991 end
992
993 # PARS: the result's match
994 def result
995   result
996 end
997
998 # PARS: the result's match
999 def result
1000   result
1001 end
```

2.4.2.7 KernelMode Rootkits: SSDT Hooks

1. Hook SSDT

Hook SSDT entry corresponding to [NTQueryDirectoryFile](#).

2. Call Function

Now, whenever the NTQueryDirectoryFile function is called, the malicious function will be called instead.

```
def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @candidates = observations - @control
  evaluate_candidates

  freeze

  # TODO: the experiment's control
end

# TODO: the name of the experiment
def experiment_name
  @experiment.name
end

# TODO: did the results a match between all
def matched?
  # ...
end

def experiment_result
  1.1
```

2.4.2.7 KernelMode Rootkits: SSDT Hooks

3. Pass Control

Right after the malicious function gets called, call the original function (NTQueryDirectoryFile) and get its results, a directory listing in this case.

4. Alter & Return Results

- Modify the results (hide a file or directory, for example)
- Pass the results back to the caller.

```
def __init__(self, experiment):
    self.experiment = experiment
    self.observations = []
    self.control = []
    self.candidates = []
    self.evaluate_candidates()

    freeze

    # TODO: the experiment's context
    def context:
        experiment.context

    # TODO: the name of the experiment
    def experiment_name:
        experiment.name

    # TODO: add the results a match between all
    def match:
        # ...
    for result in result:
        ...
```


2.4.2.8 KernelMode Rootkits: IRP Hooks

Almost everything in the Windows kernel uses IRPs, for example, the network interface (TCP, UDP, etc.), file system, keyboard, mouse, and almost all existing drivers.

Here is a little snippet from the **Microsoft WinDDK** showing how critical IRPs are:

```
DriverObject->MajorFunction[IRP_MJ_CREATE]=DiskPerfCreate;  
DriverObject->MajorFunction[IRP_MJ_READ]=DiskPerfReadWrite;  
DriverObject->MajorFunction[IRP_MJ_WRITE]=DiskPerfReadWrite;  
DriverObject->MajorFunction[IRP_MJ_SYSTEM_CONTROL]=DiskPerfWmi;  
DriverObject->MajorFunction[IRP_MJ_SHUTDOWN]=DiskPerfShutdownFlush;  
DriverObject->MajorFunction[IRP_MJ_FLUSH_BUFFERS]=DiskPerfShutdownFlush;  
DriverObject->MajorFunction[IRP_MJ_PNP]=DiskPerfDispatchPnp;  
DriverObject->MajorFunction[IRP_MJ_POWER]=DiskPerfDispatchPower;
```

2.4.2.8 KernelMode Rootkits: IRP Hooks

Each device object has its own function table.

Hooking the function pointers of such objects is called **DKOM** (**D**irect **K**ernel **O**bject **M**anipulation). The effects of this technique are global to the system.

```
def initialize(experiment, observations = [], control = null)
  @experiment = experiment
  @observations = observations
  @control = control
  @data = []
  freeze
end

def context
  experiment.context
end

def initialize_the_name_of_the_experiment
  @experiment_name =
    experiment_name
end

def match
  # Check the result a match between all
  # ...
  @experiment.result
end
```

2.4.2.8 KernelMode Rootkits: IRP Hooks

A function pointer can be hooked as follows:

```
old_power_irp=DriverObject->MajorFunction[IRP_MJ_POWER];  
DriverObject->MajorFunction[IRP_MJ_POWER]=my_new_irp;
```

2.4.2.8 KernelMode Rootkits: IRP Hooks

The basic IRP design is so that after an IRP has been created, it is passed to all the devices registered at lower levels. The design has a pre-processing mode and a post-processing mode.

Pre-processing is done when an IRP arrives, and post-processing is done when the IRP has been processed by all the levels below the current level.

```
24 def initialize(experiment, observations = [], control = null)
25   @experiment = experiment
26   @observations = observations
27   @control = control
28   @candidates = observations + [control]
29   evaluate_candidates
30
31   freeze
32 end
33
34 # PUBLIC: the experiment's control
35 def control
36   @control
37 end
38
39 # PUBLIC: the name of the experiment
40 def name
41   @experiment
42 end
43
44 # PUBLIC: get the results a match between all
45 def matches
46   @matches
47 end
48
49 # PRIVATE: result of the IRP
50 def result
51   @result
52 end
```

2.4.2.9 Userland Rootkits: IAT Hooks

IAT stands for Import Address Table, and it is used to resolve runtime dependencies.

It accomplishes this by listing what Windows API functions it needs and where they are located, within each DLL (including its address).

```
20 def initialize_experiment
21   @experiment = Experiment.new
22   @observations = []
23   @control = Control.new
24   @candidates = []
25   @evaluate_candidates
26 end
27
28 def initialize_experiment(observations = [], control = nil)
29   @experiment = experiment
30   @observations = observations
31   @control = control
32   @candidates = observations.map { |o| o.candidates }
33   @evaluate_candidates
34 end
35
36 def evaluate_candidates
37   @candidates.each do |c|
38     c.evaluate
39   end
40 end
41
42 def experiment_name
43   @experiment_name
44 end
45
46 def experiment_name=
47   @experiment_name =
48 end
49
50 def match
51   @match
52 end
53
54 def match=
55   @match =
56 end
57
58 def result
59   @result
60 end
61
62 def result=
63   @result =
64 end
65
66 def result_sub
67   @result_sub
68 end
69
70 def result_sub=
71   @result_sub =
72 end
```

2.4.2.9 Userland Rootkits: IAT Hooks

IAT Hooking involves modifying the IAT table of the executable and replacing the addresses to redirect certain functions to use the malicious function instead.

2.4.2.10 Userland Rootkits: EAT Hooks

EAT stands for Export Address Table, this table is maintained in DLLs. These files contain support functions for other executables.

The difference between IAT and EAT Hooking is that since EATs exist only in DLLs (under normal settings), most of the times EAT Hooking is utilized only in DLLs, while IAT Hooking can be done in both DLLs and EXEs.

2.4.2.11 Userland Rootkits: Inline Hooks

Inline Hooking is the most difficult. With this technique, the malware will modify the API function itself.

The malware can accomplish this by modifying the first few bytes of the target function code and replacing them with malicious code which tells EIP (instruction pointer) to execute code somewhere else in memory.

2.4.2.12 Rootkits: Process Hiding

One characteristic all rootkits have in common is hiding their existence in the target system.

When a hunter/analyst is attempting to look for malware running on a system, rootkits will typically hide running processes from view making them difficult to detect using conventional methods.

```
18 def __init__(self, experiment, observations = [], control = null):
19     self.experiment = experiment
20     self.observations = observations
21     self.control = control
22
23     # The experiment will result in 2
24     # observations - an array of Observations, or Observations
25     # control - the control observation
26
27     self.initialize(experiment, observations = [], control = null)
28
29     @experiment
30     observations = observations
31     control = control
32
33     freeze
34
35     experiment.context
36
37     experiment_name
38     experiment_name
39
40     # A class that will result a match between all
41     def match
42     def match
43     def match
44     def match
45     def match
46     def match
47     def match
48     def match
49     def match
50     def match
51     def match
52     def match
53     def match
54     def match
55     def match
56     def match
57     def match
58     def match
59     def match
60     def match
61     def match
62     def match
63     def match
64     def match
65     def match
66     def match
67     def match
68     def match
69     def match
70     def match
71     def match
72     def match
73     def match
74     def match
75     def match
76     def match
77     def match
78     def match
79     def match
80     def match
81     def match
82     def match
83     def match
84     def match
85     def match
86     def match
87     def match
88     def match
89     def match
90     def match
91     def match
92     def match
93     def match
94     def match
95     def match
96     def match
97     def match
98     def match
99     def match
100    def match
```

2.4.2.12 Rootkits: Process Hiding

Malware will hook [NtOpenProcess](#) (Native API) probably using SSDT hooking techniques.

It would also need to hide the process from the [EPROCESS](#) list. This list is maintained by the operating system for all active processes.

2.4.2.12 Rootkits: Process Hiding

Within the EPROCESS list structure, one of the important members is **ActiveProcessLinks**, which is the doubly [linked list](#) with *FLINK (Forward Link) and *BLINK (Back Link) as pointers to the other structures.

In order to hide the process, the malware will need to unlink the structure relative to the malicious and compromised process from the list. If the driver is loaded, the malware will also have to unlink it from **PsLoadedModuleList**.

```
24 void __cdecl theExperimentDataControl::
25 observations = an array of Observations, an Observations
26 * control = the control observation
27
28 void InitializeExperiment(Observations = {}, control = null)
29 {
30     @experiment = experiment
31     @observations = observations
32     @control = control
33     @candidates = observations - {control}
34     evaluate_candidates
35
36 freeze
37
38     experiment.context
39     experiment_name
40     experiment_name
41     experiment_name
42     experiment_name
43     experiment_name
44     experiment_name
45     experiment_name
46     experiment_name
47     experiment_name
48     experiment_name
49     experiment_name
50     experiment_name
51     experiment_name
52     experiment_name
53     experiment_name
54     experiment_name
55     experiment_name
56     experiment_name
57     experiment_name
58     experiment_name
59     experiment_name
60     experiment_name
61     experiment_name
62     experiment_name
63     experiment_name
64     experiment_name
65     experiment_name
66     experiment_name
67     experiment_name
68     experiment_name
69     experiment_name
70     experiment_name
71     experiment_name
72     experiment_name
73     experiment_name
74     experiment_name
75     experiment_name
76     experiment_name
77     experiment_name
78     experiment_name
79     experiment_name
80     experiment_name
81     experiment_name
82     experiment_name
83     experiment_name
84     experiment_name
85     experiment_name
86     experiment_name
87     experiment_name
88     experiment_name
89     experiment_name
90     experiment_name
91     experiment_name
92     experiment_name
93     experiment_name
94     experiment_name
95     experiment_name
96     experiment_name
97     experiment_name
98     experiment_name
99     experiment_name
100    experiment_name
```

2.4.3 Masquerading

Masquerading is defined as pretending to be someone one is not. This definition can apply to techniques malware uses to avoid detection.

At times, instead of going to great lengths to use an advanced technique, like one of ones previously mentioned, they can often opt for a very simple one, such as naming the malware similar to something that might go undetected.

```
20 def __init__(self, experiment_name, control_name, observations_name, control_name):
21     self.experiment_name = experiment_name
22     self.control_name = control_name
23     self.observations_name = observations_name
24     self.control_name = control_name
25     self.experiment_name = experiment_name
26     self.control_name = control_name
27     self.observations_name = observations_name
28     self.control_name = control_name
29     self.experiment_name = experiment_name
30     self.control_name = control_name
31     self.observations_name = observations_name
32     self.control_name = control_name
33     self.experiment_name = experiment_name
34     self.control_name = control_name
35     self.observations_name = observations_name
36     self.control_name = control_name
37     self.experiment_name = experiment_name
38     self.control_name = control_name
39     self.observations_name = observations_name
40     self.control_name = control_name
41     self.experiment_name = experiment_name
42     self.control_name = control_name
43     self.observations_name = observations_name
44     self.control_name = control_name
45     self.experiment_name = experiment_name
46     self.control_name = control_name
47     self.observations_name = observations_name
48     self.control_name = control_name
49     self.experiment_name = experiment_name
50     self.control_name = control_name
51     self.observations_name = observations_name
52     self.control_name = control_name
53     self.experiment_name = experiment_name
54     self.control_name = control_name
55     self.observations_name = observations_name
56     self.control_name = control_name
57     self.experiment_name = experiment_name
58     self.control_name = control_name
59     self.observations_name = observations_name
60     self.control_name = control_name
61     self.experiment_name = experiment_name
62     self.control_name = control_name
63     self.observations_name = observations_name
64     self.control_name = control_name
65     self.experiment_name = experiment_name
66     self.control_name = control_name
67     self.observations_name = observations_name
68     self.control_name = control_name
69     self.experiment_name = experiment_name
70     self.control_name = control_name
71     self.observations_name = observations_name
72     self.control_name = control_name
73     self.experiment_name = experiment_name
74     self.control_name = control_name
75     self.observations_name = observations_name
76     self.control_name = control_name
77     self.experiment_name = experiment_name
78     self.control_name = control_name
79     self.observations_name = observations_name
80     self.control_name = control_name
81     self.experiment_name = experiment_name
82     self.control_name = control_name
83     self.observations_name = observations_name
84     self.control_name = control_name
85     self.experiment_name = experiment_name
86     self.control_name = control_name
87     self.observations_name = observations_name
88     self.control_name = control_name
89     self.experiment_name = experiment_name
90     self.control_name = control_name
91     self.observations_name = observations_name
92     self.control_name = control_name
93     self.experiment_name = experiment_name
94     self.control_name = control_name
95     self.observations_name = observations_name
96     self.control_name = control_name
97     self.experiment_name = experiment_name
98     self.control_name = control_name
99     self.observations_name = observations_name
100    self.control_name = control_name
```

2.4.3 Masquerading

They might call the malware `svch0st`, `scvhost`, `svchost32`, etc. You can find a list of different variations used to masquerade malware as `svchost.exe` [here](#).

Also, placing the malware in `C:\Windows` or `C:\Windows\System32` directories is another way of masquerading as a legitimate executable or dll.

2.4.3 Masquerading

Malware will also hide in other locations, such as in:

- Temporary folders
- Temporary Internet files
- Program Files

This doesn't exclude other locations, such as the Recycle Bin.

```
21 # @param: EXPERIMENT - the experiment to be run
22 # @param: OBSERVATIONS - an array of Observations, in which
23 # @param: CONTROL - the control Observation
24 # @param: EVALUATE - the function to evaluate the candidates
25
26 def initialize(experiment, observations = [], control = null)
27   @experiment = experiment
28   @observations = observations
29   @control = control
30   @candidates = observations - [control]
31   evaluate_candidates
32
33   freeze
34 end
35
36 # @param: the experiment's context
37 def context
38   @context ||= experiment.context
39 end
40
41 # @param: the result of a match between an observation and a candidate
42 def match
43   @match ||= nil
44 end
45
46 # @param: the result of a match between an observation and a candidate
47 def match?
48   @match ||= nil
49 end
50
51 # @param: the result of a match between an observation and a candidate
52 def match!(result)
53   @match = result
54 end
```

2.4.4 Packing / Compression

A packer is software which compresses an executable. They were initially designed to decrease the size of executable files.

However, malware authors recognized very quickly that decreasing the file size will decrease the number of patterns in the file, so it will decrease the chance of detection by AV products.

2.4.4 Packing / Compression

Some malware authors have gone to the trouble of creating their own packers (such as Yoda Packer), while others use readily available packers (such as [UPX](https://upx.github.io/)).

2.4.5 Recompiling

As we should know, an executable will have a signature, such as a MD5 hash, that will identify it.

With recompiling, especially with different compilers, the malware authors are looking to avoid detection because a different hash will be produced.

```
20 def initialize_experiment
21   @experiment = Experiment.new
22   @observations = []
23   @control = Control.new
24   @candidates = []
25   @evaluate_candidates
26 end
27
28 def initialize_experiment(observations = [], control = nil)
29   @experiment = Experiment.new
30   @observations = observations
31   @control = control
32   @candidates = observations.map { |o| o.candidates }
33   @evaluate_candidates
34 end
35
36 def evaluate_candidates
37   @candidates.each do |c|
38     c.evaluate
39   end
40 end
41
42 def experiment_name
43   @experiment_name
44 end
45
46 def experiment_name= name
47   @experiment_name = name
48 end
49
50 def match?
51   @match
52 end
53
54 def match= result
55   @match = result
56 end
```



2.4.6 Obfuscation

The next two techniques (**obfuscation** and **anti-reversing**) address the point that once the malware has been found, it attempts to remain irreversible.

2.4.6 Obfuscation

Code obfuscation techniques transform/change a program in order to make it more difficult to analyze while preserving the original functionality. Code obfuscation is used both by malware and legitimate software to protect itself.

The difference is that malware uses it to either prevent detection or to make reverse engineering more difficult. Malware obfuscates itself every time it infects a new host, which makes it harder for a detector to recognize it.

2.4.7 Anti-Reversing Techniques

There are several methods that are used by malware to detect that analysis is being conducted on the code, or simply to misdirect the malware analyst and take him for a loop, thus increasing the time required to analyze the code.

Some of these the anti-reversing techniques are:

- Detecting that malware is being run in a virtual machine
- Detecting that a debugger is attached to the malware
- Junk code can be inserted into the malware as misdirection

Malware Persistence



2.5 Malware Persistence

As mentioned previously, there are many goals of malware once on a host machine, as noted in MITRE's ATT&CK [wiki](https://attack.mitre.org/wiki/).

In this section, we'll look at mechanisms used by malware to achieve [persistence](#).

```
38 def initialize(experiment, observations = [], control = None):
39     @experiment = experiment
40     @observations = observations
41     @control = control
42     @candidates = observations + [control]
43     evaluate_candidates
44
45 freeze
46
47 def experiment_context
48     @experiment.context
49 end
50
51 # Return the name of the experiment
52 def experiment_name
53     @experiment.name
54 end
55
56 # Return the result a match between all
57 def matched?
58     @experiment.result == 1
59 end
```

2.5.1 Autostart Locations

Malware uses these autostart locations to survive reboots. By adding an entry into one of many autostart locations, either within the registry or specific folders, the program will execute when the user logs in.

The most common, and the most obvious, registry location is:

HKLM\Software\Microsoft\Windows\CurrentVersion\Run.

2.5.1 Autostart Locations

Here are a few other **common** locations:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run
- HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run

2.5.1 Autostart Locations

Some **not so common** locations include:

- HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon\Userinit
- HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options
- HKLM\Software\Wow6432Node\Windows NT\CurrentVersion\Image File Execution Options

2.5.1 Autostart Locations

To get an idea of all the locations that malware can use within the registry or folders, look at the [AutoRuns](#) tool from [Sysinternals](#).

You may also want to refer to ATT&CK's wiki page [here](#).

<https://docs.microsoft.com/en-us/sysinternals/downloads/autoruns>

<https://docs.microsoft.com/en-us/sysinternals/>

<https://attack.mitre.org/wiki/Technique/T1060>

2.5.2 Scheduled Task

Utilities such as **at.exe** and **schtasks.exe** can be used to schedule execution of malware or scripts based on a specific date/time or even an event.

An adversary can use task scheduling to execute malware or scripts on startup or on a schedule, not only for persistence, but for other tasks as well, such as privilege escalation.

```
20 def initialize(experiment, observations = [], control = null)
21   # Initialize the experiment
22   # @experiment: the experiment object
23   # @observations: an array of Observations, or nil
24   # @control: the control observation
25
26   # Initialize the experiment
27   @experiment = experiment
28   @observations = observations
29   @control = control
30   @candidates = observations - [control]
31   evaluate_candidates
32
33   # Return the experiment object
34   @experiment
35
36   # Return the result of the experiment
37   @result
38
39   # Return the result of the experiment
40   @result
41
42   # Return the result of the experiment
43   @result
44
45   # Return the result of the experiment
46   @result
47
48   # Return the result of the experiment
49   @result
50
51   # Return the result of the experiment
52   @result
53
54   # Return the result of the experiment
55   @result
56
57   # Return the result of the experiment
58   @result
59
60   # Return the result of the experiment
61   @result
62
63   # Return the result of the experiment
64   @result
65
66   # Return the result of the experiment
67   @result
68
69   # Return the result of the experiment
70   @result
71
72   # Return the result of the experiment
73   @result
74
75   # Return the result of the experiment
76   @result
77
78   # Return the result of the experiment
79   @result
80
81   # Return the result of the experiment
82   @result
83
84   # Return the result of the experiment
85   @result
86
87   # Return the result of the experiment
88   @result
89
90   # Return the result of the experiment
91   @result
92
93   # Return the result of the experiment
94   @result
95
96   # Return the result of the experiment
97   @result
98
99   # Return the result of the experiment
100  @result
```

2.5.2 Scheduled Task

An example from ATT&CK for scheduled tasks is from [APT3](#):

- **`schtasks /create /tn "mysc" /tr C:\Users\Public\test.exe /sc ONLOGON /ru "System"`**

2.5.3 COM Hijacking

“The Microsoft Component Object Model (COM) is a platform-independent, distributed, object-oriented system for creating binary software components that can interact. COM is the foundation technology for Microsoft's OLE (compound documents), ActiveX (Internet-enabled components), as well as others.” – as referenced by Microsoft [here](https://docs.microsoft.com/en-us/windows/win32/com/the-component-object-model).

2.5.3 COM Hijacking

In a nutshell, it enables interaction between software components through the Windows operating system.

Adversaries can use this system (COM) to insert malicious code that can be executed instead of legitimate software by hijacking the COM references. You can read the ATT&CK document [here](https://attack.mitre.org/wiki/Technique/T1122).

2.5.4 DLL Hijacking: Search Order

In this technique, the malware is attacking a feature of the Windows operating system. When an executable runs, there is a search order to find the required DLL for the executable.

The first location that it will look in is within the local directory and eventually, it will look in the C:\Windows\System32 folder as well, if the executable is still searching for the required DLL.

2.5.4 DLL Hijacking: Search Order

You can read more about:

- The technique on ATT&CK [here](#).
- DLL Library Search Order [here](#).

```
10 def initialize(experiment, observations = [], control = null)
11   @experiment = experiment
12   @observations = observations
13   @control = control
14   @candidates = observations + [control]
15   evaluate_candidates
16
17   freeze
18 end
19
20 # Returns the experiment's context
21 def context
22   @experiment.context
23 end
24
25 # Returns the name of the experiment
26 def experiment_name
27   @experiment.name
28 end
29
30 # Returns the result a match between all candidates
31 def matches
32   @experiment.result
33 end
```

2.5.5 DLL Hijacking: Phantom DLL

This technique also attacks a feature of the Windows operating system. Executables may try to load old DLLs that no longer exist on modern Windows operating systems.

Attackers can place a malicious DLL and name it to what the executable is looking for, then the malicious DLL will be loaded.

2.5.5 DLL Hijacking: Phantom DLL

Please reference a very resourceful blog on Hexacorn.com. He has a series titled, **Beyond Good Ol' Run Key**, where some parts of the series are dedicated to Phantom DLLs.

You can read more about this technique on his blog in one of the postings [here](http://www.hexacorn.com/blog/2016/06/02/beyond-good-ol-run-key-part-40/).

2.5.6 DLL Hijacking: Side Loading

This technique uses the **WinSxS** (Windows side-by-side) folder. This Windows folder (or feature) is used by applications to prevent problems which may arise due to updated and duplicated versions of DLLs. As you can imagine, a malicious DLL can be placed into this folder.

Please reference the information on ATT&CK about this technique [here](#). You can read more about this technique in the following FireEye PDF called [DLL Side-Loading: A Thorn in the Side of the Anti Virus Industry](#).

2.5.7 Windows Services: Service Creation

Service creation has been a popular technique to achieve persistence. A malicious service can hide in plain sight among many other services. A service can be configured to run at boot, which will often load before AV software. Services are created using the built-in Windows “sc” command.

The following link, [here](#), is an old post, but still holds true to the fundamentals to use service creation for malicious purposes.

2.5.8 Windows Services: Service Replacement

With service replacement, attackers will look for an existing service to replace with their malware. If it's not configured to auto start, then that setting can be tweaked. With a low privilege account, the attacker can look for services with poor ACL configurations to replace the executable with the malware.

2.5.9 Windows Services: Service Recovery

Another technique an attacker can accomplish using Services is to set up recovery actions when a service fails. With this technique, an attacker can configure the service to run an executable, such as malware, upon failure.

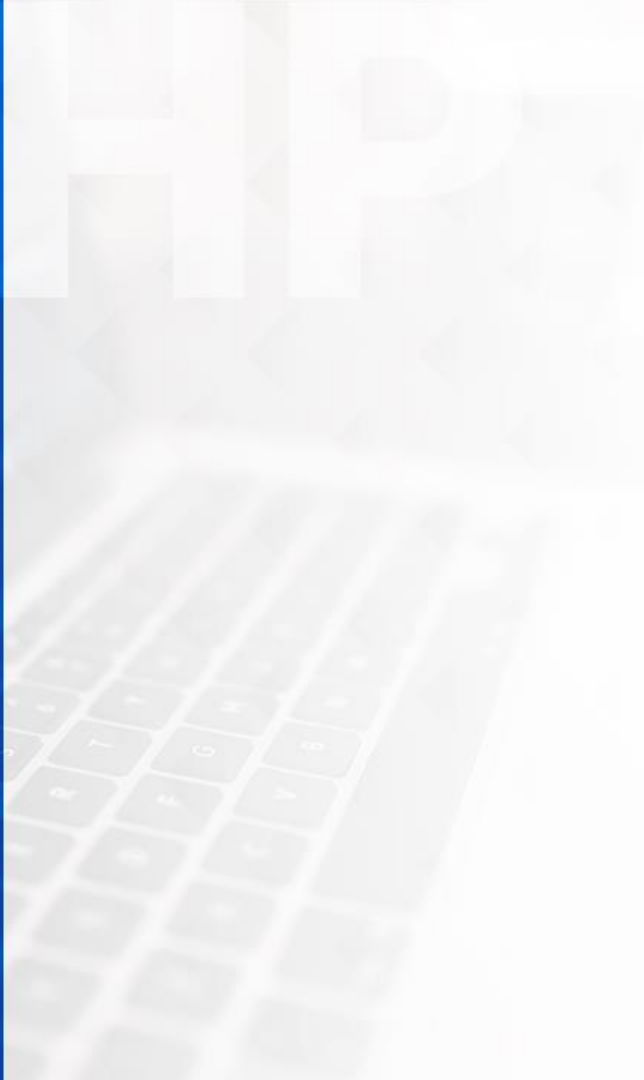
You can read more about this feature [here](#).

Module Conclusion

This concludes the module on malware. We have covered:

- Different classifications of malware.
- How malware infects the host system.
- Different techniques malware uses to evade detection and maintain persistence.

```
21 def initialize(experiment, observations = [], control = null)
22   @experiment = experiment
23   @observations = an Array of Observations, an Array
24   @control = the control Observation
25
26   def initialize(experiment, observations = [], control = null)
27     @experiment = experiment
28     @observations = observations
29     @control = control
30     @candidates = observations - [control]
31     evaluate_candidates
32
33   freeze
34 end
35
36 experiment.context
37
38 # TODO: the name of the experiment
39 def experiment_name
40   experiment.name
41 end
42
43 # TODO: get the result a match between all
44 def matches?
45   ..
46   @candidates/result.rb 11
```



References



References

[Blue Pill](#)

<http://theinvisiblethings.blogspot.com/2006/06/introducing-blue-pill.html>

[SubVirt](#)

<https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/subvirt.pdf>

[USB Rubber Ducky](#)

<http://usbrubberducky.com/#!index.md>

[USBdriveby](#)

<http://samy.pl/usbdriveby/>



References

[Teensy](https://www.pjrc.com/teensy/)

<https://www.pjrc.com/teensy/>

[BadUSB](https://nakedsecurity.sophos.com/2014/10/06/badusb-now-with-do-it-yourself-instructions/)

<https://nakedsecurity.sophos.com/2014/10/06/badusb-now-with-do-it-yourself-instructions/>

[URL Shortener](https://goo.gl/)

<https://goo.gl/>

[Exploit Kits](https://blog.malwarebytes.com/threat-analysis/2017/03/exploit-kits-winter-2017-review/)

<https://blog.malwarebytes.com/threat-analysis/2017/03/exploit-kits-winter-2017-review/>

References

[Watering Hole Attack](#)

<https://www.trendmicro.com/vinfo/us/threat-encyclopedia/web-attack/137/watering-hole-101>

[Malvertising](#)

<https://arstechnica.com/information-technology/2016/12/millions-exposed-to-malvertising-that-hid-attack-code-in-banner-pixels/>

[Malvertising Example](#)

<https://www.forbes.com/sites/leemathews/2017/08/03/over-a-million-coders-targeted-by-chrome-extension-hack/#670c91059c9d>

[ATT&CK Technique Matrix](#)

https://attack.mitre.org/wiki/Technique_Matrix



References

[ATT&CK \(Evading Defenses\)](https://attack.mitre.org/wiki/Defense_Evasion)

https://attack.mitre.org/wiki/Defense_Evasion

[ATT&CK \(Execution\)](https://attack.mitre.org/wiki/Execution)

<https://attack.mitre.org/wiki/Execution>

[ATT&CK \(Escalate Privileges\)](https://attack.mitre.org/wiki/Privilege_Escalation)

https://attack.mitre.org/wiki/Privilege_Escalation

[ATT&CK \(Steal Credentials\)](https://attack.mitre.org/wiki/Credential_Access)

https://attack.mitre.org/wiki/Credential_Access



References

[ATT&CK \(Exfiltration\)](https://attack.mitre.org/wiki/Exfiltration)

<https://attack.mitre.org/wiki/Exfiltration>

[ATT&CK \(Persistence\)](https://attack.mitre.org/wiki/Persistence)

<https://attack.mitre.org/wiki/Persistence>

[Alternate Data Streams](https://docs.microsoft.com/en-us/archive/blogs/askcore/alternate-data-streams-in-ntfs)

<https://docs.microsoft.com/en-us/archive/blogs/askcore/alternate-data-streams-in-ntfs>

[CreateFile](https://docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea)

<https://docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-createfilea>



References

[WriteFile](https://docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefile)

<https://docs.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefile>

[Streams \(tool\)](https://docs.microsoft.com/en-us/sysinternals/downloads/streams)

<https://docs.microsoft.com/en-us/sysinternals/downloads/streams>

[Sysinternals Suite](https://docs.microsoft.com/en-us/sysinternals/downloads/)

<https://docs.microsoft.com/en-us/sysinternals/downloads/>

[Get-Item](https://docs.microsoft.com/en-us/powershell/module/Microsoft.PowerShell.Management/Get-Item?view=powershell-5.1)

<https://docs.microsoft.com/en-us/powershell/module/Microsoft.PowerShell.Management/Get-Item?view=powershell-5.1>



References

[Get-Content](#)

<https://docs.microsoft.com/en-us/powershell/module/Microsoft.PowerShell.Management/Get-Content?view=powershell-5.1>

[PowerShell v5](#)

<https://docs.microsoft.com/en-us/powershell/scripting/whats-new/what-s-new-in-windows-powershell-50?view=powershell-7>

[CoinMiner](#)

<https://www.microsoft.com/en-us/wdsi/threats/malware-encyclopedia-description?Name=Trojan:Win32/CoinMiner>

[CoinMiner Analysis](#)

<https://secreary.com/ReversingMalware/CoinMiner/>



References

[Process Injection](https://attack.mitre.org/wiki/Technique/T1055)

<https://attack.mitre.org/wiki/Technique/T1055>

[CreateToolhelp32Snapshot](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489(v=vs.85).aspx)

[Process32First](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684834(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684834\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684834(v=vs.85).aspx)

[Process32Next](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684836(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684836\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684836(v=vs.85).aspx)



References

[GetModuleHandle](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms683199\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms683199(v=vs.85).aspx)

[GetProcAddress](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms683212\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms683212(v=vs.85).aspx)

[OpenProcess](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684320\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684320(v=vs.85).aspx)

[VirtualAllocEx](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa366890(v=vs.85).aspx)



References

[WriteProcessMemory](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx)

[CreateRemoteThread](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682437\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682437(v=vs.85).aspx)

[LoadLibrary](#)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684175\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684175(v=vs.85).aspx)

[NTCreateThreadEx](#)

<https://undocumented.ntinternals.net/index.html?page=UserMode/Undocumented%20Functions/NT%20Objects/Thread/NtCreateThread.html>

References

[RtlCreateUserThread](https://undocumented.ntinternals.net/index.html?page=UserMode/Undocumented%20Functions/Executable%20Images/RtlCreateUserThread.html)

<https://undocumented.ntinternals.net/index.html?page=UserMode/Undocumented%20Functions/Executable%20Images/RtlCreateUserThread.html>

[Reflective DLL Injection](https://github.com/stephenfewer/ReflectiveDLLInjection)

<https://github.com/stephenfewer/ReflectiveDLLInjection>

[Thread32First](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686728(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms686728\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686728(v=vs.85).aspx)

[Thread32Next](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686731(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms686731\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686731(v=vs.85).aspx)



References

[OpenThread](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684335(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms684335\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms684335(v=vs.85).aspx)

[SuspendThread](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686345(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms686345\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms686345(v=vs.85).aspx)

[ResumeThread](https://msdn.microsoft.com/en-us/library/windows/desktop/ms685086(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms685086\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms685086(v=vs.85).aspx)

[Peering Inside the PE](https://msdn.microsoft.com/en-us/library/ms809762.aspx)

<https://msdn.microsoft.com/en-us/library/ms809762.aspx>



References

[Process Hollowing](https://attack.mitre.org/wiki/Technique/T1093)

<https://attack.mitre.org/wiki/Technique/T1093>

[CreateProcess](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682425(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682425\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682425(v=vs.85).aspx)

[ZwUnmapViewOfSection](https://msdn.microsoft.com/ru-ru/library/ff567119(v=vs.85).aspx)

[https://msdn.microsoft.com/ru-ru/library/ff567119\(v=vs.85\).aspx](https://msdn.microsoft.com/ru-ru/library/ff567119(v=vs.85).aspx)

[NtUnmapViewOfSection](https://msdn.microsoft.com/en-us/library/ff557711(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/ff557711\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/ff557711(v=vs.85).aspx)



References

[SetThreadContext](https://msdn.microsoft.com/en-us/library/windows/desktop/ms680632(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms680632\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms680632(v=vs.85).aspx)

[SetWindowsHookEx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms644990(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms644990\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms644990(v=vs.85).aspx)

[Hook Types](https://msdn.microsoft.com/en-us/library/windows/desktop/ms644959(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms644959\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms644959(v=vs.85).aspx)

[NTQueryDirectoryFile](https://msdn.microsoft.com/en-us/library/windows/hardware/ff556633(v=vs.85).aspx) (ZwQueryDirectoryFile)

[https://msdn.microsoft.com/en-us/library/windows/hardware/ff556633\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff556633(v=vs.85).aspx)



References

[Windows Driver Development Kit](#)

<https://docs.microsoft.com/en-us/windows-hardware/drivers/download-the-wdk>

[Direct Kernel Object Manipulation](#)

<https://www.blackhat.com/presentations/win-usa-04/bh-win-04-butler.pdf>

[NTOpenProcess](#)

[https://msdn.microsoft.com/en-us/library/windows/hardware/ff556571\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff556571(v=vs.85).aspx)

[EPROCESS List](#)

[https://msdn.microsoft.com/en-us/library/windows/hardware/ff544273\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff544273(v=vs.85).aspx)



References

[ActiveProcessLinks \(Linked List\)](#)

[https://msdn.microsoft.com/en-us/library/windows/hardware/ff554296\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff554296(v=vs.85).aspx)

[Svchost Abuse](#)

<http://www.hexacorn.com/blog/2013/07/04/the-typographical-and-homomorphic-abuse-of-svchost-exe/>

[UPX Packer](#)

<https://upx.github.io/>

[AutoRuns \(tool\)](#)

<https://docs.microsoft.com/en-us/sysinternals/downloads/autoruns>



References

[APT3 \(group\)](https://attack.mitre.org/wiki/Group/G0022)

<https://attack.mitre.org/wiki/Group/G0022>

[COM Hijacking](https://attack.mitre.org/wiki/Technique/T1122)

<https://attack.mitre.org/wiki/Technique/T1122>

[DLL Search Order Hijacking](https://attack.mitre.org/wiki/Technique/T1038)

<https://attack.mitre.org/wiki/Technique/T1038>

[DLL Search Order \(MSDN\)](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682586(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682586\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682586(v=vs.85).aspx)



References

[Phantom DLLs](#)

<http://www.hexacorn.com/blog/2016/06/02/beyond-good-ol-run-key-part-40/>

[DLL Side Loading](#)

<https://attack.mitre.org/wiki/Technique/T1073>

[DLL Side-Loading \(FireEye Report\)](#)

<https://www.fireeye.com/content/dam/fireeye-www/global/en/current-threats/pdfs/rpt-dll-sideload.pdf>

[Service Creation](#)

<https://www.bleepingcomputer.com/tutorials/how-malware-hides-as-a-service/>



References

[Service Recovery Actions](https://technet.microsoft.com/en-us/library/cc753662(v=ws.11).aspx)

[https://technet.microsoft.com/en-us/library/cc753662\(v=ws.11\).aspx](https://technet.microsoft.com/en-us/library/cc753662(v=ws.11).aspx)

[COM Object Model](https://docs.microsoft.com/en-us/windows/win32/com/the-component-object-model)

<https://docs.microsoft.com/en-us/windows/win32/com/the-component-object-model>

[CreateToolhelp32Snapshot](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682489(v=vs.85).aspx)

[WriteProcessMemory](https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx)

[https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms681674(v=vs.85).aspx)



References

[Windows Sysinternals](https://docs.microsoft.com/en-us/sysinternals/)

<https://docs.microsoft.com/en-us/sysinternals/>

[Registry Run Keys / Startup Folder](https://attack.mitre.org/wiki/Technique/T1060)

<https://attack.mitre.org/wiki/Technique/T1060>

